

Sabrina’s Game: From Unsampleability to Metaheuristic Optimisation

Kevin Gur

VU University Amsterdam, The Netherlands
`t.k.guer@student.vu.nl`

Abstract. This paper studies Sabrina’s Game, in which every cell of an $N \times N$ integer grid must be paired with an orthogonal neighbor to form a valid solution. Each pairing has an objective value, the absolute difference between the two paired cells, and the total objective value of a valid solution is the sum of all pairing objective values, which is to be minimized. This problem suffers from unsampleability: generating a valid random solution becomes increasingly difficult as the grid size gets larger, to the point where it is no longer feasible. This problem was addressed by adding a single heuristic to the random and greedy algorithms without fundamentally changing their randomness, at least for the grid sizes tested in this paper. With a valid initial solution now within reach for every grid size, we turn to the second part of the problem: minimizing the total objective value. For this, three metaheuristics are applied: hill climbing, simulated annealing, and plant propagation. Two different mutation operators are tested for each algorithm under a common evaluation budget. Across all grid sizes, the hybrid mutation operator consistently outperforms the simple flip mutation, and simulated annealing proves to be the strongest performing metaheuristic algorithm overall across both mutation operators.

Keywords: Sabrina’s Game · Random sampling · Unsampleability · Metaheuristics.

1 Introduction

Sabrina’s Game originates from the MSc thesis of Sabrina Strijker [1], a former University of Amsterdam student. Her project involved randomly pairing each cell of a square grid with one of its neighboring cells. Generating such pairings randomly makes it difficult to produce a valid solution because a random attempt often leaves some cells unpaired. This difficulty of randomly producing valid solutions becomes even more pronounced as the grid size grows.

Grids in this problem are $N \times N$ sized where every cell has a randomly assigned number in the range $[0,100)$. The cell count of the grid must be an even number in order to get a valid solution. Otherwise, there will always be a cell left out. The objective is to generate a valid solution and then optimize the solution by minimizing the total objective value. In Fig.1a) an initial grid

where no pairing has yet been done can be seen. A solution is valid only if every cell is paired with a neighboring cell and no cells are left unpaired, as shown in Fig 1b). An example of an invalid solution could be seen in Fig.1c) where there are cells that are not paired with a neighboring cell and are left out. Even if one cell is 'stuck', meaning the cell does not have a neighbor left to pair up with, it is directly an invalid solution. The total objective value is only defined for valid solutions. It can only be computed when every cell has a pair.

Every pairing has an objective value, calculated as the absolute difference between the selected cell and the neighboring cell that it is paired with:

$$|cell - neighbor| \quad (1)$$

Total objective value is the sum of the objective values of each pairing, calculated as follows:

$$\sum |cell - neighbor| \quad (2)$$

At first, generating a random pairing solution seems easy, one could simply pair each cell with a randomly chosen neighbor. In practice, however, this approach often fails. As cells are paired one by one, the process can reach a state where a remaining cell has no available neighbor left to pair with, making it an invalid solution, a situation referred to as the 'stuck problem' throughout this paper. When the grid size gets larger the stuck problem happens more frequently, and the probability of producing a valid solution such as in Fig.1b) by random construction drops sharply with the grid size getting larger.

This difficulty is an example of what van den Berg [2] calls unsampleability, meaning the lack of an efficient way to randomly generate valid solutions. Sabrina's Game shares this property with several problems for which no polynomial time uniform random sampling algorithm is known, such as the traveling tournament problem [3] and the HP protein folding problem [8, 9]. In these problems, most randomly generated solutions are invalid, which limits metaheuristic algorithms that depend on random initialisation and mutation [2].

In this thesis, this problem is addressed in two parts. First, we mitigate unsampleability by introducing a single heuristic to the random and greedy algorithms without altering the nature of either algorithm. Second, once a valid solution is obtained, we minimize its total objective value using three metaheuristics: hill climbing, simulated annealing, and plant propagation. Two different mutation operators are implemented, and each algorithm is tested with both.

75	2	28	14	26	12	30	82
86	87	2	32	19	72	83	57
64	69	83	52	53	45	58	73
22	18	68	56	6	52	27	48
86	40	7	74	6	83	7	37
32	89	89	18	71	29	99	56
92	65	23	69	28	99	92	79
73	68	95	85	56	94	46	67

(a) Initial grid

75	2	28	14	26	12	30	82
86	87	2	32	19	72	83	57
64	69	83	52	53	45	58	73
22	18	68	56	6	52	27	48
86	40	7	74	6	83	7	37
32	89	89	18	71	29	99	56
92	65	23	69	28	99	92	79
73	68	95	85	56	94	46	67

(b) Valid solution

75	2	28	14	26	12	30	82
86	87	2	32	19	72	83	57
64	69	83	52	53	45	58	73
22	18	68	56	6	52	27	48
86	40	7	74	6	83	7	37
32	89	89	18	71	29	99	56
92	65	23	69	28	99	92	79
73	68	95	85	56	94	46	67

(c) Invalid solution

Fig. 1: (a) An 8×8 instance of Sabrina's Game: the initial grid (b) a valid solution where every cell is paired (c) an invalid solution with unpaired cells.

2 Methods

2.1 Constructive Algorithms

Constructive algorithms build a valid solution starting from an initial grid instance, adding one pairing at a time. This section describes the random and greedy algorithms and the unstuck heuristic that helps them reach a valid solution.

2.1.1 Random and Greedy

The random and greedy algorithms build a solution until every cell has been paired. At each step, an available cell is selected randomly and paired with one of its available neighbors. After the pairing, both cells are marked as used and removed from the available cells so they cannot be considered for the next

pairing. The two algorithms only differ in how they choose the neighbor cell. The random algorithm pairs the selected cell with a randomly chosen available neighbor. The greedy algorithm, on the other hand, calculates the objective value for every available neighbor cell pairing and selects the one that gives the least objective value. We can also say that it selects the neighbor that is closest in value to the selected cell. That means the randomness in the greedy algorithm is only when selecting the initial cell. As described in the introduction, this process for both algorithms does not guarantee finding a valid solution. If an algorithm ends up with an invalid solution, it restarts the process, and each invalid solution is counted as a failed attempt. The algorithm repeats this process until a valid solution is found, which is then recorded as the result of a single run.

2.1.2 The Unstuck Heuristic: Random Unstuck and Greedy Unstuck

This section introduces a single heuristic that solves the stuck problem for the vast majority, even for larger grids. This heuristic works without altering the core nature of the random or greedy algorithm. Applied to the random and greedy algorithms, we refer to the resulting variants as random unstuck and greedy unstuck, respectively.

The heuristic changes only how the next cell is selected, not how its neighbor is chosen. Instead of selecting the initial cell to pair at random, we choose the cell with the fewest available neighbors first. If multiple different cells have the same smallest number of available neighbors, one of them is chosen at random. After the initial cell is selected this way, the neighbor is still chosen exactly as before for both the random and greedy algorithms, where random selects uniformly at random, and greedy selects by lowest objective value. The idea behind this heuristic came after analyzing the grids pairing step by step while the algorithms were trying to find a valid solution. A cell with only one or two available neighbors is at the greatest risk of becoming stuck. Cells with many available neighbors are left for later, since they have more neighbor options to find a valid pair.

2.2 Mutation Operators

Two mutation operators are used throughout this thesis for the metaheuristic algorithms: a local flip mutation and a hybrid mutation. Both mutations operate on valid solutions that are gathered from random unstuck and greedy unstuck algorithms. This allows the metaheuristics in the following section to explore the valid solution space. Both mutation operators work on small square blocks of cells. To determine if a block can be used, each cell is labelled with a special id of its pairing, so that the paired cells share the same id. A block is only usable if every pairing id inside the block appears exactly twice, meaning no pairing crosses the block boundary. After each mutation, the available blocks and windows are updated, since changing the orientations can create new valid blocks or windows and remove others.

2.2.1 Flip Mutation

The flip mutation operates on 2×2 blocks on the grid. Each block must only have two pairings to be called a 'flippable block'. In such a case both of the pairings have to be horizontal or vertical. The mutation selects one of the flippable blocks randomly and switches its orientation. If the pairings are horizontal, they become vertical and the other way around.

The block objective value is computed by summing the objective values of the two pairings for the old block and the new block after the flip mutation is applied. If the new block objective value is lower than the old block objective value, the mutation is kept, meaning the mutation improved the solution. Otherwise, the block is flipped back to its old orientation.

The flip only changes the arrangement inside of a single 2×2 block. All cells outside of this block remain paired, so the resulting solution is guaranteed to remain valid.

2.2.2 Hybrid Mutation

The hybrid mutation allows larger moves in the solution space than the flip mutation alone. It works with a 4×4 window in the grid and the flip mutation together. For the 4×4 window to be a valid window, it should have only eight pairings inside of it. The set of all valid windows is referred to as the 'available windows', from which the mutation selects randomly.

With a 0.3 probability, a random 4×4 window is selected from the available windows, and the window objective value is calculated. After this, eight pairings are entirely discarded. Then the random unstuck algorithm is used to repair the cells as described in Section 2.1.2. The resulting change is calculated by comparing the window objective value before and after this repairing process.

If no 4×4 window is available, or with the remaining 0.7 probability, the hybrid mutation uses the flip mutation described in Section 2.2.1. The reason for giving a smaller probability to the 4×4 window mutation than the flip mutation is because in small grids there may be only one valid 4×4 window available. If the 4×4 window mutation were applied every time such a window existed, the same window would be repaired repeatedly, which would eliminate exploration.

By combining a small local change with a larger change, the hybrid mutation is expected to explore the solution space more broadly than only doing flip mutations, at the cost of larger and less predictable steps in total objective value.

2.3 Metaheuristics

Metaheuristics are high level, problem independent strategies used to find good solutions to hard optimization problems [10]. In this paper metaheuristic algorithms take the solutions produced by the random unstuck and greedy unstuck algorithms and are used to optimize these solutions, meaning improving them by minimizing their total objective value. All three metaheuristics start from the

same solution. Throughout each run, the best solution found so far is stored and updated whenever a better solution is found. The reason for this is that some of these algorithms can make worsening moves and reach a worse solution during the search, so the solution at the end of a run is not always the best that has been found. The stored best is returned at the end. Each algorithm runs until the same evaluation budget is reached. For hill climbing and simulated annealing, one evaluation equals one mutation, for plant propagation, it is equal to one offspring created. In both cases, one evaluation corresponds to producing and evaluating one solution, which makes the comparison consistent.

2.3.1 Hill Climbing

Hill climbing is a simple local search metaheuristic [7]. In our problem it searches for better solutions by moving toward lower total objective values, since the goal is to minimize it. Starting from a single valid solution, it repeatedly applies a mutation. After each mutation, the new total objective value is compared to the previous one. If a mutation results in a lower total objective value, we accept the mutation since a lower total objective value is an improvement.

A mutation resulting in an equal total objective value is also accepted. The reason for this is that equal neighboring values indicate a plateau in the search space. A plateau is a flat area of the search space where no uphill or downhill exists, meaning the neighboring solutions have the same values. Accepting equal values is referred to as a sideways move in this paper. Without sideways moves, the search could not move on the plateaus, and might fail to reach better solutions that could be found. A mutation is only rejected when it results in a higher total objective value, meaning it is a worsening move. The main limitation of the hill climbing algorithm is that once it reaches a local minimum, it becomes stuck, since it never accepts a worsening move.

2.3.2 Simulated Annealing

Simulated annealing is inspired by the physical annealing process, in which a material is first melted and then cooled slowly to reach a stable state. Cooling too quickly results in a defective structure that is only locally optimal [4]. Unlike hill climbing, simulated annealing can escape local minima by occasionally accepting worsening moves. After each mutation, a move that improves the total objective value or equals the total objective value is always accepted. However, a worsening move is not immediately rejected. It is accepted with a probability based on the Metropolis criterion [12], which depends on the increase in the total objective value and on the current temperature. The probability is given by the following:

$$\exp(-\Delta/T) \tag{3}$$

where Δ is the increase in the total objective value and T is the current temperature. A larger Δ or a lower temperature results in a lower acceptance probability of worsening moves.

At the start of the search, the temperature is high, so many worsening moves are accepted. This allows the search to explore the solution space. At each evaluation, the temperature decreases by multiplying the temperature by a fixed factor α rather than remaining constant for a number of steps. This corresponds to the inhomogeneous form of simulated annealing [11].

$$T \leftarrow T \cdot \alpha \quad (4)$$

The cooling factor α is not chosen randomly but found from the evaluation budget so that the temperature decreases from its starting value T_{start} to its minimum value T_{min} over exactly the given number of evaluations:

$$\alpha = \left(\frac{T_{min}}{T_{start}} \right)^{1/n_{eval}} \quad (5)$$

As the temperature decreases, the probability of accepting worsening moves also decreases. The algorithm shifts toward only accepting improving moves, similar to hill climbing.

2.3.3 Plant Propagation

The plant propagation algorithm is based on how plants spread to survive. It is a nature inspired algorithm that is used for complex optimisation problems. It is inspired by how strawberry plants propagate through runners [5]. A plant wants to find a good spot where there are enough resources. A plant in a good spot sends out more short runners to exploit its close area, while a plant in a poor spot sends out fewer long runners to explore better ground that is farther away.

Hill climbing and simulated annealing each improve a single solution. The plant propagation algorithm, on the other hand, is a population based metaheuristic that works with multiple solutions at once. Since all metaheuristics start from a single valid solution, an initial population must be created for the plant propagation algorithm. This population is created by making N copies of the starting solution and applying a fixed number of mutations to each copy. This way, the copies are diversified, turning one starting solution into a population of distinct solutions.

In this problem, a good spot is equal to a good solution, and a bad spot is equal to a worse solution. The length of a runner corresponds to the number of mutations applied to produce an offspring. A short runner means few mutations, keeping the offspring close to its parent, while a long runner means many mutations, moving the offspring farther away in the solution space. According to this, a good solution produces many offspring with few mutations each, and a poor solution produces few offspring with many mutations each, corresponding to exploitation and exploration, respectively.

To determine how good each solution is, every solution is given a fitness score. The total objective values are normalized to a scale between 0 and 1. Since the goal is to minimize the total objective value, a solution with a low total objective

value receives a fitness score closer to 1. A solution with a high total objective value receives a score closer to 0. So every solution is ranked on this scale based on its total objective value. The fitness score of a solution is calculated as follows:

$$f_i = \frac{O_{max} - O_i}{O_{max} - O_{min}} \quad (6)$$

where O_i is the total objective value of solution i , and O_{max} and O_{min} are the highest and lowest total objective values in the current population, respectively.

The fitness score of each solution is then used to determine how many offspring and how many mutations it will have. A solution with fitness f produces

$$n_{runners} = \max(1, \text{round}(n_{max} \cdot f \cdot r)) \quad (7)$$

runners, where n_{max} is the maximum number of runners that it could have and r is a random number between $[0, 1)$.

Then each runner applies a number of mutations calculated as:

$$n_{mutation} = \max(1, \text{round}((1 - f) \cdot m_{max} \cdot r)) \quad (8)$$

where m_{max} is the maximum number of mutations. This number scales together with the grid size. The random r brings variation, so that solutions with the same fitness score do not always have the same number of runners or mutations.

As mentioned before, higher fitness scored solutions will have more runners but fewer mutations. Lower fitness scored solutions will have fewer runners but more mutations.

After the offspring are created and their mutations are applied, we gather all offspring and the parents into a pool, and the best N solutions are selected to form the next generation. This cycle of creating offspring and selecting the best N is called one generation. Since each generation uses the solutions from the previously generated generation, copying one solution is only done once, at the start.

3 Experimental Setup

For this experiment, grids of five sizes were used: 6×6 , 8×8 , 12×12 , 16×16 , 18×18 . Cell values are drawn randomly from $[0, 100)$, and 50 different grids were generated per grid size. The random and greedy algorithms could not produce a valid solution for the 16×16 and 18×18 grids due to the unsampleability problem, so they were only run on the three smaller sizes. The random unstuck and greedy unstuck algorithms solved this problem, and they were run on all five grid sizes.

For each grid, every algorithm was run 30 times, giving 1500 valid solutions per algorithm per grid size. For the unstuck algorithms, this results in 7500 valid solutions each across all grid sizes.

The metaheuristic algorithms were run on all valid solutions gathered from the random unstuck and greedy unstuck algorithms. Each metaheuristic algorithm was given an evaluation budget of 10,000. This number was chosen based

on a convergence analysis of the largest grid size (18×18), averaged over 30 runs, shown in Figure 2. Using both mutation operators, all algorithms were observed to converge well before the budget. With the flip mutation, convergence occurs by around 5,000 evaluations, and with the hybrid mutation, by around 7,000 at the latest, with hill climbing converging much earlier in both cases, at around 2,000. The evaluation budget of 10,000 provides a comfortable margin well after all algorithms have converged.

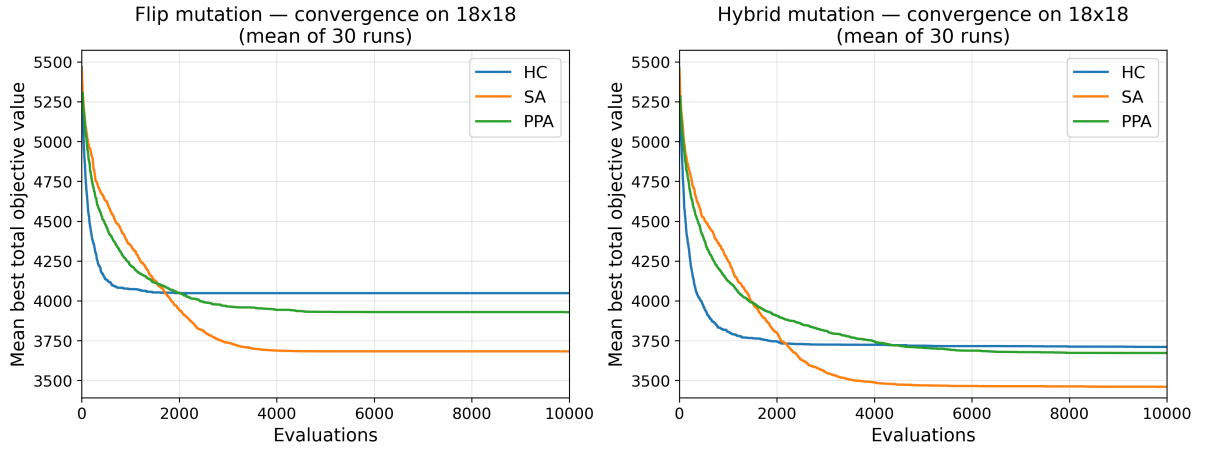


Fig. 2: Convergence of the three metaheuristics on the 18×18 grid, for the flip mutation (left) and the hybrid mutation (right), averaged over 30 runs.

Hill climbing has no parameters that need to be set. However, simulated annealing and plant propagation require several parameters to be set.

For simulated annealing, the starting temperature is 100, and the minimum temperature, at which the algorithm stops, is 0.01. As described in Section 2.3.2, there is a cooling factor α that controls how the temperature decreases from the starting temperature to the minimum temperature value. α is set to 0.99908 so that the temperature reaches its minimum exactly at the end of the evaluation budget.

For plant propagation, the population size N is 20, meaning 20 copies are made from the single starting solution at the beginning of the process. Each copy is diversified with 5 initial mutations, and the maximum number of runners is set to 5. The population size ($N=20$) and maximum number of runners (5) were chosen within the effective range identified by De Jonge and van den Berg [6]. They found that plant propagation performs better within a certain window of these values, while the specific values within that window are largely interchangeable.

The comparison is evaluation based. The three algorithms use different processes to reach a solution, and giving each algorithm the same evaluation budget allows the algorithms to be compared on equal terms.

4 Results

This section presents the results of our experiments. The results first show how unsampleability affects the constructive algorithms and how the unstuck heuristic addresses it, followed by a comparison of the three metaheuristics under each mutation operator, and finally a comparison between the two mutation operators.

4.1 Unsampleability and the Unstuck Heuristic

Figure 3 shows the mean number of failed attempts before a valid solution is found for all four constructive algorithms across all grid sizes, on a logarithmic scale. As can be seen for the random and greedy algorithms, even on the smallest grid size, some failed attempts occur before finding a valid solution. We can say that for the grid sizes 6×6 and 8×8 , it is at a manageable level. However, when it comes to the grid size of 12×12 , the number of failed attempts drastically increases to approximately 196,000 for the random algorithm and around 483,000 for the greedy algorithm. For the two bigger grid sizes, they could not find a valid solution within a reasonable time. This is the unsampleability problem. As for the random unstuck and greedy unstuck algorithms, it can be seen that for the bigger grid sizes, they have a failed attempt count around 1. From these results, it can be said that the unstuck heuristic addressed the unsampleability problem for the grid sizes that were used in this paper.

Figure 4 shows that the solution quality of the random and greedy algorithms is preserved after adding the unstuck heuristic. The greedy and greedy unstuck algorithms produce lower total objective values than the random algorithm because they always pair each cell with the neighbor that gives the least objective value, which directly minimizes the objective value of each pairing and therefore the total objective value.

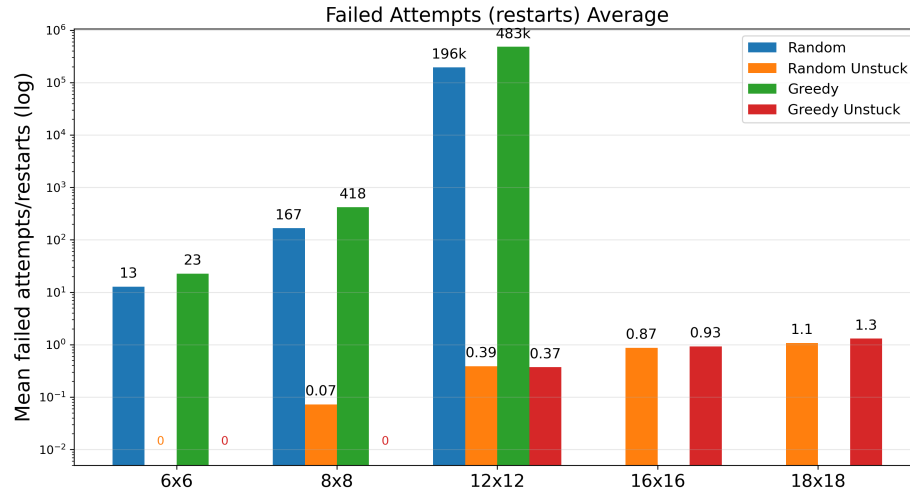


Fig. 3: Mean number of failed attempts before a valid solution is found, for the constructive algorithms across all grid sizes (logarithmic scale)

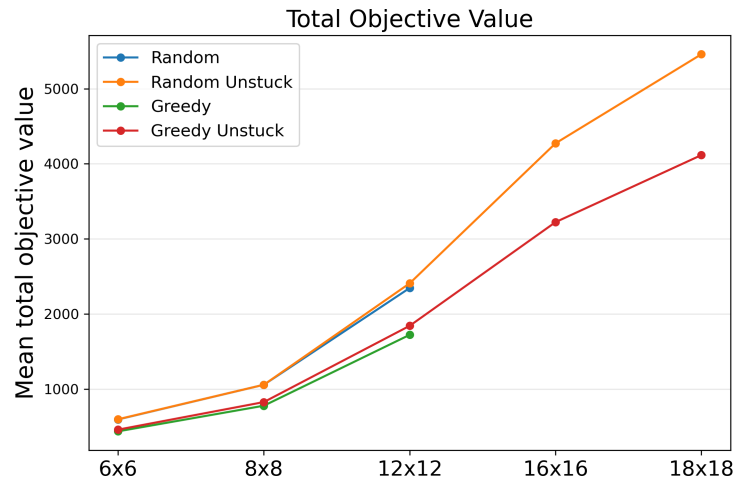


Fig. 4: Mean total objective value of the solutions produced by the constructive algorithms across all grid sizes

4.2 Flip Mutation

The left graph of Figure 5 shows the final total objective value for each algorithm. For every grid size, simulated annealing has the lowest total objective value, making it the best of the three. Hill climbing has the highest total objective value for every grid size, making it the worst. Plant propagation falls in between. On the smaller grids, the total objective values are close to each other. As the grid size gets larger, simulated annealing clearly reaches lower objective values. However, on the 18×18 grid, plant propagation (3877) is much closer to hill climbing (3631) than to simulated annealing (3631).

The right graph of Figure 5 shows the mean improvement over the initial solution. For every grid size, simulated annealing achieves the highest mean improvement, while hill climbing achieves the lowest, with plant propagation lying in between. Simulated annealing improves the initial solution by around 23% at every grid size, whereas the improvement of hill climbing is around 17% and plant propagation decreases as the grid gets larger.

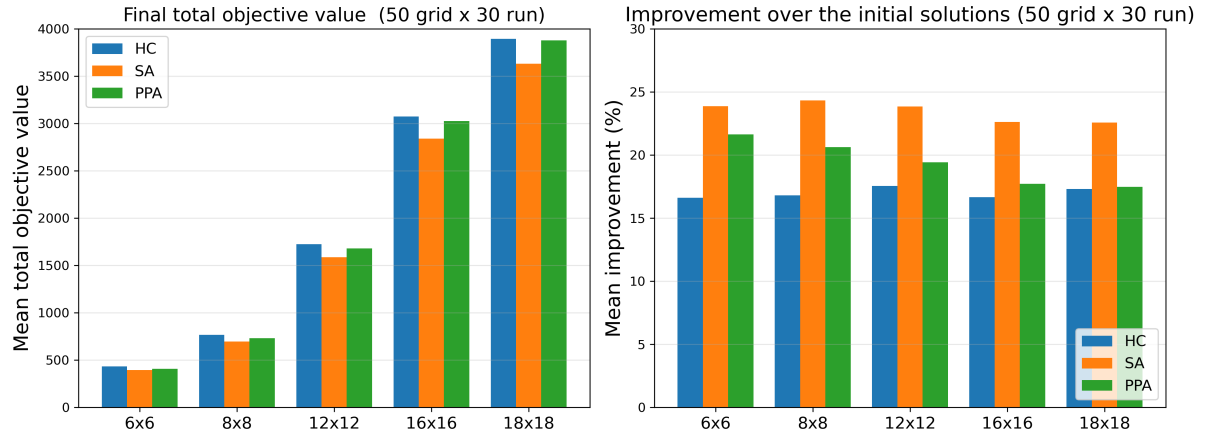


Fig. 5: Comparison of the three metaheuristics using the flip mutation across all grid sizes, for the initial solutions produced by the random unstuck and greedy unstuck algorithms. The left graph shows the mean final total objective value, and the right graph shows the mean improvement over the initial solution.

4.3 Hybrid Mutation

The left graph of Figure 6 shows the final total objective value for each algorithm. We see the same story as the flip mutation. For every grid size, simulated annealing has the lowest total objective value, making it the best of the three. Hill climbing has the highest total objective value for every grid size, making it the worst, with plant propagation lying in between. On the smaller grids, the total objective values of the three algorithms are close to each other. As the grid size increases, simulated annealing clearly reaches lower objective values. On the 18×18 grid, plant propagation (3635) is again much closer to hill climbing (3645) than to simulated annealing (3463).

The right graph of Figure 6 shows the mean improvement over the initial solution. For every grid size, simulated annealing achieves the highest mean improvement, while hill climbing the lowest, with plant propagation lying in between. Simulated annealing improves the initial solution by around 26% at every grid size, whereas the improvement of hill climbing is around 22% and plant propagation around 23%.

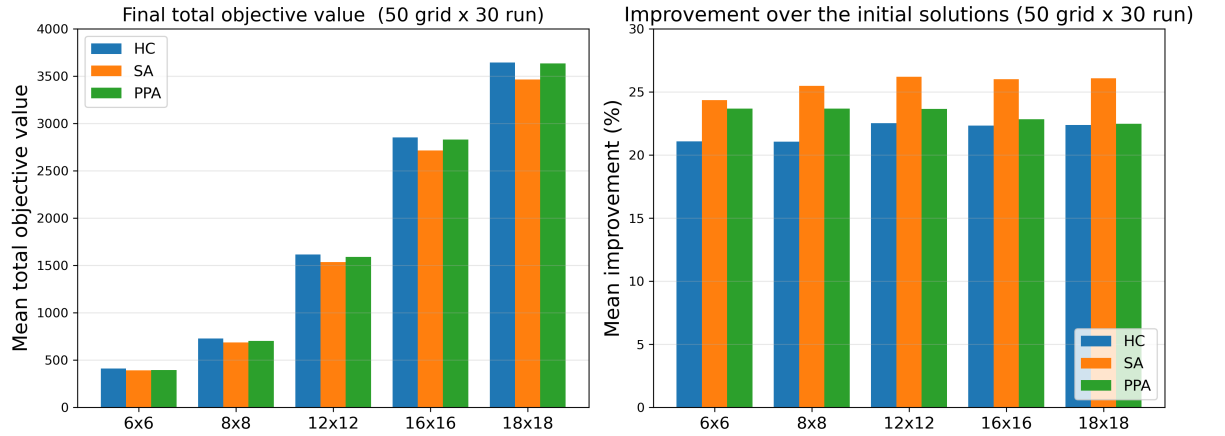


Fig. 6: Comparison of the three metaheuristics using the hybrid mutation across all grid sizes, for the initial solutions produced by the random unstuck and greedy unstuck algorithms. The left graph shows the mean final total objective value, and the right graph shows the mean improvement over the initial solution.

4.4 Flip vs Hybrid Mutation

Figure 7 shows the mean improvement over the initial solution for each metaheuristic for both the flip and hybrid mutation across all grid sizes. For all three algorithms, the hybrid mutation improves the initial solution more than the flip mutation. For simulated annealing the mean improvement increases from 23% with the flip mutation to 26% with the hybrid mutation. For hill climbing the mean improvement increases from 17% to 22%. Lastly for plant propagation, the mean improvement increases from 20% to 23%. Hill climbing shows the largest gain when switching from the flip to the hybrid mutation.

Figure 8 shows a head to head comparison of the flip and hybrid mutation operators for each metaheuristic, over 15,000 improved solutions that start from the same initial solution. Each bar shows the percentage of cases in which the hybrid mutation is better, the two mutation operators are equal, or the flip mutation is better. For all three algorithms, the hybrid mutation is better in the majority of cases: 81% for hill climbing, 74% for plant propagation and 63% for simulated annealing. In every case, the percentage where the two mutation operators are equal is higher than the percentage where the flip mutation is better. Both figures show that hybrid mutation clearly outperforms the flip mutation.

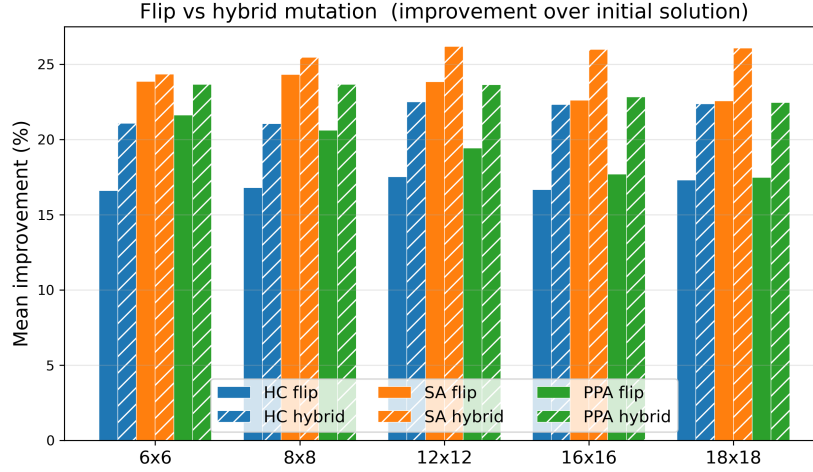


Fig. 7: Mean improvement over the initial solution for each metaheuristic, using the flip and hybrid mutation operators, across all grid sizes.

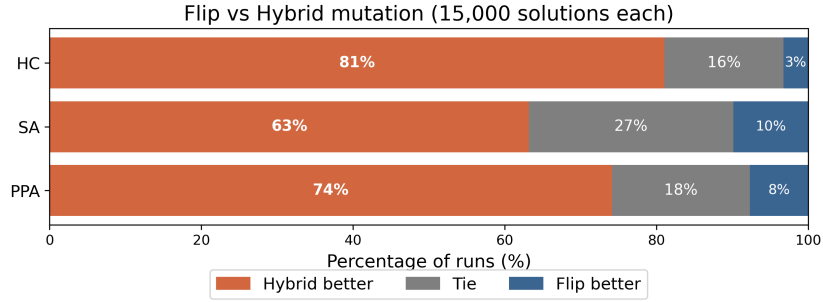


Fig. 8: Head to head comparison of the flip and hybrid mutation operators for each metaheuristic, over 15,000 paired solutions. Each bar shows the percentage of cases in which the hybrid mutation is better, the two are equal, or the flip mutation is better.

5 Conclusion

This paper studied Sabrina's Game, in which every cell of an $N \times N$ integer grid must be paired with an orthogonal neighbor cell to form a valid solution, with the goal of minimizing the total objective value. Generating valid solutions randomly becomes increasingly difficult as the grid size gets larger, to the point where valid solutions can no longer be found. This paper approached the problem in two parts: first, addressing the unsampleability problem, and then optimizing the valid solutions using metaheuristics.

By adding a single heuristic to the random and greedy algorithms, unsampleability was largely addressed without changing the nature of the algorithms or the quality of the solutions. Almost no failed attempts occurred even for the largest grids, which provided the metaheuristics with a valid initial solution for every grid size. The three metaheuristics were compared using both mutation operators. The same pattern was seen for both mutations. Simulated annealing performed best, hill climbing performed worst, and plant propagation fell in between. The two mutation operators were also compared. The hybrid mutation was found to be consistently better than the flip mutation for every algorithm, in the majority of cases.

6 Discussion

Simulated annealing being the best performing algorithm could be because it escapes local minima by accepting worsening moves with a probability. Hill climbing, on the other hand, was expected to perform worse than simulated annealing, because it only accepts better or equally good moves, which causes it to get stuck in local minima. Plant propagation lies in between the two. Its solution quality is close to hill climbing, and on the 18×18 grid it is the closest.

The hybrid mutation makes larger moves in the search space compared to the flip mutation because of its 4×4 window repairing. Hill climbing is the algorithm that benefits the most from this mutation, which may be because it is the simplest of the three and therefore gains the most from a stronger mutation operator.

The results of this paper are limited to the grid sizes that were tested. Larger grid sizes could show different results. In addition, the results from the meta-heuristic algorithms were not compared to optimal solutions, so it is not known how far they are from the optimum.

In future work, larger grids could be tested. Plant propagation may benefit from its population based nature on larger grids, and it could be possible it outperforms simulated annealing on very large grid sizes. Comparing the results to optimal solutions would also be valuable. Finally, additional mutation operators could be added and compared with the existing ones.

References

1. Strijker, S.: Understanding collusive bribery dynamics: Agent-based modeling in a social network context (MSc thesis) (2025), <https://dspace.uba.uva.nl/server/api/core/bitstreams/c6c16fd9-7571-4361-9e58-2f83dc3eccd8/content>
2. van den Berg, D.: Can Sabrina's Game be solved by Evolutionary Algorithms? (2026)
3. Loyen, B., Bart, D., Richoux, F., van den Berg, D.: The traveling tournament problem: Valid solutions are different across instance sizes. 17th International Joint Conference on Computational Intelligence (IJCCI 2025)
4. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. *Science* **220**(4598), 671–680 (1983)
5. Salhi, A., Fraga, E.S.: Nature-inspired optimisation approaches and the new plant propagation algorithm. International Conference on Numerical Analysis and Optimization (2011)
6. De Jonge, M., van den Berg, D.: Plant propagation parameterization: offspring & population size. *Evo* Late-Breaking Abstracts* (2020)
7. Russell, S., Norvig, P.: *Artificial Intelligence: A Modern Approach*, 4th edn., pp. 120–129. Pearson (2021)
8. Horn, R., van den Berg, D., Jansen, R., Verduin, K., van Eck, O.: Randomly sampling HP-protein conformations: Mission impossible? In: Bäck, T., et al. (eds.) *IJCCI 2023*, SCI 1196, pp. 323–338. Springer (2025)
9. Jansen, R., Horn, R., van Eck, O., Verduin, K., Thomson, S.L., van den Berg, D.: Can HP-protein folding be solved with genetic algorithms? Maybe not. In: *Proceedings of the 15th International Joint Conference on Computational Intelligence (IJCCI 2023)* (2023)
10. Sörensen, K.: Metaheuristics—the metaphor exposed. *International Transactions in Operational Research* **22**(1), 3–18 (2015)
11. van Laarhoven, P.J.M., Aarts, E.H.L.: *Simulated Annealing: Theory and Applications*. Mathematics and Its Applications, vol. 37. Springer, Dordrecht (1987)
12. Metropolis, N., Rosenbluth, A.W., Rosenbluth, M.N., Teller, A.H., Teller, E.: Equation of state calculations by fast computing machines. *The Journal of Chemical Physics* **21**(6), 1087–1092 (1953)